

Ocelo

Arquitetura do banco de dados

Sugestão de arquitetura para levar o catálogo de protocolos dos arquivos TypeScript para o PostgreSQL: diagrama de relacionamentos, tabelas, índices, decisões, fluxos de dados, operação e como escalar.

Versão 1.0.0
Novembro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 **Contexto** **4**

1.1 Onde os dados estão hoje 4

1.2 Quando vale migrar 4

1.3 Princípios do modelo 4

02 **Diagrama de relacionamentos** **5**

2.1 Entidades 5

2.2 Relacionamentos 6

03 **Tabelas** **7**

3.1 versao 7

3.2 protocolo 7

3.3 protocolo_texto 7

3.4 no, ligacao e faixa 8

3.5 legenda 8

3.6 passo e fato 8

3.7 pacote 9

3.8 usuario e progresso (opcionais) 9

04 **Índices e consultas** **11**

4.1 Índices 11

4.2 Consultas principais 11

05 **Decisões de arquitetura** **12**

5.1 PostgreSQL 12

5.2 Textos bilíngues em jsonb 12

5.3 Quadros em jsonb 12

5.4 Versões imutáveis 12

5.5 O site continua estático 12

06 **Fluxos de dados** **13**

6.1 Edição e publicação 13

6.2 Leitura no site 13

6.3 Migrando os dados atuais 13

07 **Operação e escala** **14**

7.1 Backup e recuperação 14

Copyright (©) 2026 Erik Pimenta

7.2 Segurança 14
7.3 Como escalar 14
7.4 Migrações 14

Copyright (©) 2026 Erik Pimenta

01 Contexto

1.1 Onde os dados estão hoje

Na versão 1.0.0 o Ocelo não usa banco de dados. Os 33 protocolos são arquivos TypeScript em `src/dados/protocolos`, validados pelos testes e empacotados no build estático. As preferências de cada pessoa (idioma, tema, paleta, vista e velocidade) ficam no `localStorage` do navegador.

Esse desenho é o certo para o volume atual: o catálogo muda pouco, é público, cabe em 160 kB comprimidos e a hospedagem estática no GitHub Pages não tem custo nem superfície de ataque.

1.2 Quando vale migrar

- **Edição sem deploy:** professores ou uma equipe de conteúdo precisam criar e revisar protocolos por uma interface, com rascunho e publicação.
- **Contas e progresso:** turmas querem acompanhar quem concluiu cada protocolo.
- **Mais idiomas** e traduções feitas por pessoas diferentes.
- **Catálogo grande** (centenas de protocolos ou variações), em que carregar tudo no primeiro acesso deixa de compensar.

> **RECOMENDAÇÃO**
> Enquanto nenhum desses pontos existir, mantenha os arquivos TypeScript. Quando um deles aparecer, siga este documento: o modelo foi desenhado para receber exatamente os dados que já existem, sem perder a validação.

1.3 Princípios do modelo

- PostgreSQL como fonte da verdade, com integridade garantida por chaves e restrições, não pela aplicação.
- Conteúdo bilíngue em `jsonb` com as chaves de idioma, validado por restrição.
- A especificação de cada quadro em `jsonb`, porque ela já é validada pelo motor de quadros e muda de forma conforme o protocolo.
- Versões publicadas imutáveis: o site lê sempre uma versão fechada, e a edição acontece num rascunho.
- O site continua estático: o banco gera um JSON por versão, servido por CDN.

Copyright (©) 2026 Erik Pimenta

02 Diagrama de relacionamentos

2.1 Entidades

protocolo	
id PK	text
sigla	text
categoria	enum
camada	smallint
ordem	smallint
versao_id FK	bigint

protocolo_texto	
protocolo_id PK FK	text
idioma PK	char(2)
nome	text
resumo	text
descricao	text
observacao	text

fato	
id PK	bigint
protocolo_id FK	text
passo_id FK	bigint
ordem	smallint
rotulo	jsonb
valor	jsonb

no	
protocolo_id PK FK	text
id PK	text
dispositivo	enum
nome	jsonb
ip	inet
mac	macaddr
x, y	numeric

ligacao	
protocolo_id PK FK	text
origem PK FK	text
destino PK FK	text

faixa	
protocolo_id PK FK	text
ordem PK	smallint
no_id FK	text

legenda	
protocolo_id PK FK	text
tipo PK	text
nome	jsonb
cor	enum

passo	
id PK	bigint
protocolo_id FK	text
ordem	smallint
titulo	jsonb
resumo	jsonb
descricao	jsonb

pacote	
id PK	bigint
passo_id FK	bigint
ordem	smallint
de, para FK	text
proto	text
tipo FK	text
dt_ms	integer
perdido	boolean
quadro	jsonb

versao	
id PK	bigint
numero	text
publicada_em	timestamptz
autor	text
estado	enum

progresso	
usuario_id PK FK	uuid
protocolo_id PK FK	text
ultimo_passo	smallint
concluido_em	timestamptz

usuario	
id PK	uuid
email	citext
idioma	char(2)
preferencias	jsonb
criado_em	timestamptz

Copyright (©) 2026 Erik Pimenta

Um **protocolo** pertence a uma **versão**, tem textos por idioma, fatos, nós, ligações, faixas, legenda e passos. Cada **passo** tem fatos e pacotes; cada **pacote** referencia os nós de origem e destino e um tipo da legenda. **Usuário** e **progresso** são opcionais e só existem se houver contas.

2.2 Relacionamentos

De	Para	Cardinalidade	Regra
versao	protocolo	1 : N	Apagar uma versão publicada é proibido; rascunhos apagam em cascata
protocolo	protocolo_texto	1 : N	Um registro por idioma; pt e en obrigatórios
protocolo	no, ligacao, faixa, legenda	1 : N	Em cascata com o protocolo
protocolo	passo	1 : N	Ordem única por protocolo
passo	pacote	1 : N	Ordem única por passo
passo / protocolo	fato	1 : N	Um fato pertence a um passo ou à visão geral, nunca aos dois
pacote	no	N : 1 (duas vezes)	Origem e destino precisam existir no mesmo protocolo e estar nas faixas
pacote	legenda	N : 1	O tipo precisa existir na legenda do protocolo
usuario	progresso	1 : N	Apagar a conta apaga o progresso

Copyright (©) 2026 Erik Pimenta

03 Tabelas

3.1 versao

Campo	Tipo	Restrições	Descrição
id	bigint	PK, identity	Identificador da versão do catálogo
numero	text	NOT NULL, UNIQUE	Versão semântica, como 1.0.0
estado	enum versao_estado	NOT NULL, 'rascunho' ou 'publicada'	Só rascunhos podem ser editados
publicada_em	timestamptz	NULL até publicar	Momento da publicação
autor	text	NOT NULL	Quem publicou

3.2 protocolo

Campo	Tipo	Restrições	Descrição
id	text	PK (com versao_id), CHECK (id ~ '^[a-z0-9-]+$')	Usado no endereço #/id
versao_id	bigint	PK, FK versao(id)	Versão a que pertence
sigla	text	NOT NULL	Sigla exibida (HTTP, Wi-Fi, TLS/SSL)
categoria	enum categoria	'core', 'aplicacao' ou 'seguranca'	Aba da lista
camada	smallint	CHECK (camada BETWEEN 1 AND 7)	Camada OSI
ordem	smallint	NOT NULL, UNIQUE (versao_id, ordem)	Posição na lista

3.3 protocolo_texto

Campo	Tipo	Restrições	Descrição
protocolo_id, versao_id	text, bigint	PK, FK protocolo	Protocolo
idioma	char(2)	PK, CHECK (idioma IN ('pt','en'))	Idioma do texto
nome, resumo, descricao	text	NOT NULL, CHECK (length(trim(x)) > 0)	Textos da lista e da visão geral

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Restrições	Descrição
observacao	text	NULL	Nota da legenda do fluxo de pacotes

3.4 no, ligacao e faixa

Campo	Tipo	Restrições	Descrição
no.id	text	PK (com protocolo)	Identificador local, como 'cliente'
no.dispositivo	enum dispositivo	12 valores	Desenho isométrico
no.nome	jsonb	CHECK (nome ? 'pt' AND nome ? 'en')	Nome bilíngue
no.ip, no.mac	inet, macaddr	NULL	Endereços quando existirem
no.x, no.y	numeric(5,2)	CHECK x 6–94, y 12–88	Posição no palco
ligacao.origem, destino	text	PK, FK no; CHECK (origem <> destino)	Ligação sem direção
faixa.ordem, no_id	smallint, text	PK (protocolo, ordem); UNIQUE (protocolo, no_id)	Ordem das faixas na sequência

3.5 legenda

Campo	Tipo	Restrições	Descrição
tipo	text	PK (com protocolo)	Chave usada pelos pacotes, como 'dns'
nome	jsonb	Bilíngue	Texto da legenda
cor	enum cor	'anil', 'oliva', 'ouro', 'brasa', 'neutra'	Token de cor da paleta

3.6 passo e fato

Campo	Tipo	Restrições	Descrição
passo.id	bigint	PK, identity	Identificador
passo.ordem	smallint	UNIQUE (protocolo, ordem)	Ordem do passo (3 a 7 por protocolo)

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Restrições	Descrição
passo.titulo, resumo, descricao	jsonb	Bilíngue	Textos do passo
fato.protocolo_id / passo_id	text / bigint	CHECK (num_nonnulls(protocolo_id, passo_id) = 1)	Dono do fato
fato.rotulo, valor	jsonb	Bilíngue	Linha da tabela de fatos

3.7 pacote

Campo	Tipo	Restrições	Descrição
id	bigint	PK, identity	Identificador
passo_id, ordem	bigint, smallint	FK passo; UNIQUE (passo_id, ordem)	Posição dentro do passo
de, para	text	FK no do mesmo protocolo; CHECK (de <> para)	Origem e destino
proto	text	NOT NULL	Coluna Protocolo da tabela
tipo	text	FK legenda do mesmo protocolo	Cor e legenda
rotulo, info, resumo, nota	jsonb	Bilíngue; resumo e nota opcionais	Textos da seta, da tabela e do cartão
dt_ms	integer	CHECK (dt_ms >= 0)	Intervalo para o pacote anterior
perdido	boolean	NOT NULL DEFAULT false	Pacote que não chega
quadro	jsonb	NOT NULL, validado por JSON Schema	Especificação do quadro (enlace, rede, portas, flags, carga)

3.8 usuario e progresso (opcionais)

Campo	Tipo	Restrições	Descrição
usuario.id	uuid	PK DEFAULT gen_random_uuid()	Identificador
usuario.email	citext	UNIQUE	Login
usuario.preferencias	jsonb	Validado	Idioma, tema, paleta, vista e velocidade
progresso.ultimo_passo	smallint	NOT NULL	Último passo visto

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Restrições	Descrição
progresso.concluido_em	timestamptz	NULL	Quando terminou o protocolo

-

Copyright (©) 2026 Erik Pimenta

04 Índices e consultas

4.1 Índices

Índice	Para quê
protocolo (versao_id, ordem)	Montar a lista na ordem certa
protocolo (versao_id, categoria, camada)	Contagens das abas e filtro por camada
passo (protocolo_id, versao_id, ordem)	Ler os passos de um protocolo
pacote (passo_id, ordem)	Ler os pacotes de um passo
protocolo_texto GIN (to_tsvector(nome || resumo))	Busca por texto no editor de conteúdo
progresso (usuario_id)	Painel de progresso da pessoa

4.2 Consultas principais

```
-- catálogo publicado, na ordem da lista
SELECT p.id, p.sigla, p.categoria, p.camada, t.nome, t.resumo
FROM protocolo p
JOIN protocolo_texto t ON t.protocolo_id = p.id AND t.versao_id = p.versao_id AND t.idioma =
$1
WHERE p.versao_id = (SELECT id FROM versao WHERE estado = 'publicada' ORDER BY publicada_em
DESC LIMIT 1)
ORDER BY p.ordem;

-- passos e pacotes de um protocolo
SELECT s.ordem AS passo, k.ordem, k.de, k.para, k.proto, k.tipo, k.dt_ms, k.perdido, k.quadro
FROM passo s JOIN pacote k ON k.passo_id = s.id
WHERE s.protocolo_id = $1 AND s.versao_id = $2
ORDER BY s.ordem, k.ordem;
```

Copyright (©) 2026 Erik Pimenta

05 Decisões de arquitetura

5.1 PostgreSQL

Relacional, com chaves compostas, restrições `CHECK`, `jsonb` indexável, tipos de rede nativos (`inet` e `macaddr`) e maturidade de backup e réplica. Um SQLite serviria para um editor local, mas o PostgreSQL atende o editor e a publicação com o mesmo modelo.

5.2 Textos bilíngues em jsonb

Cada texto curto vira um objeto `{"pt": ..., "en": ...}` com restrição que exige as duas chaves. Os textos longos do protocolo ficam em `protocolo_texto`, uma linha por idioma, o que facilita revisar e acrescentar idiomas sem mudar o esquema.

5.3 Quadros em jsonb

A especificação de cada quadro varia muito (TCP, UDP, ICMP, 802.11, EAPOL, campos da carga). Normalizar tudo em tabelas criaria dezenas de tabelas raramente consultadas. Guardar em `jsonb` validado por JSON Schema mantém a flexibilidade, e o motor de quadros continua sendo quem monta e confere os bytes.

5.4 Versões imutáveis

Editar sempre um rascunho e publicar uma versão fechada garante que o site nunca mostra conteúdo pela metade e permite voltar para a versão anterior em segundos. Cada versão publicada gera um arquivo `catalogo-<versao>.json`.

5.5 O site continua estático

O navegador não consulta o banco. Na publicação, um job exporta a versão para JSON, roda a mesma validação dos testes atuais e envia o arquivo para a CDN junto com o build. Assim a leitura escala sem custo e o banco fica protegido atrás da área de edição.

Copyright (©) 2026 Erik Pimenta

06 Fluxos de dados

6.1 Edição e publicação

1. A pessoa editora cria uma versão em rascunho copiando a última publicada.
2. Edita protocolos, passos e pacotes na área de edição; cada gravação passa pelas restrições do banco.
3. Pede a publicação. Um job exporta o rascunho para JSON e roda a validação completa (a mesma de `validacao.test.ts`) e a montagem de todos os quadros.
4. Se tudo passar, a versão vira `publicada`, o JSON vai para a CDN e o site passa a apontar para ele.
5. Se algo falhar, o rascunho continua editável e o relatório mostra o protocolo e o item com problema.

6.2 Leitura no site

O site baixa o índice do catálogo (siglas, nomes, categorias e camadas) e, ao abrir um protocolo, o arquivo daquele protocolo. Com cache longo e nomes de arquivo por versão, cada pessoa baixa cada protocolo uma vez só.

6.3 Migrando os dados atuais

1. Exporte os 33 protocolos para JSON a partir dos arquivos TypeScript (o mesmo formato do tipo `Protocolo`).
2. Crie a versão 1.0.0 e importe o JSON numa transação, tabela por tabela, respeitando a ordem das chaves estrangeiras.
3. Confira as contagens: 33 protocolos, 177 passos e 432 pacotes.
4. Gere o JSON publicado e compare com o original byte a byte.

Copyright (©) 2026 Erik Pimenta

07 Operação e escala

7.1 Backup e recuperação

- `pg_dump` diário em formato custom, guardado fora do servidor, com retenção de 30 dias.
- WAL contínuo para recuperação a um ponto no tempo, se houver edição intensa.
- Os JSON publicados já são um backup do conteúdo e podem reconstruir o catálogo.

7.2 Segurança

- Papel só de leitura para o job de exportação e papel de escrita só para a área de edição.
- Row Level Security em `progresso` e `usuario`: cada pessoa só lê e grava o que é dela.
- Conexões com TLS e segredos fora do repositório (variáveis de ambiente ou cofre).
- Auditoria de quem publicou cada versão na tabela `versao`.

7.3 Como escalar

Cenário	Estratégia
Mais acessos ao site	Nada muda no banco: a leitura é estática pela CDN
Mais protocolos	Arquivos por protocolo e carregamento sob demanda; índices já cobrem as consultas
Mais idiomas	Novas linhas em protocolo_texto e novas chaves nos textos jsonb, com a restrição ampliada
Muitas contas e turmas	Particionar progresso por mês, réplicas de leitura para relatórios

7.4 Migrações

Cada mudança de esquema vira um arquivo SQL numerado (`001_esquema.sql`, `002_indices.sql`…), aplicado em ordem dentro de uma transação e registrado numa tabela de controle, como no Bento.

Copyright (©) 2026 Erik Pimenta